

HTML + веб-сервери



Основи HTML

Введення в HTML

- HTML розшифровується як HyperText Markup Language.
- Це стандартна мова розмітки для створення веб-сторінок.
- HTML визначає структуру та представлення вмісту у Всесвітній павутині.
- Він складається з серії тегів, які визначають елементи на веб-сторінці.

Історія та еволюція HTML

- HTML вперше створив Тім Бернерс-Лі в 1990 році.
- Початкова версія, HTML 1.0, забезпечувала простий спосіб структурування документів.
- HTML 2.0, випущений у 1995 році, представив нові функції та стандартизував
- HTML 3.2, випущений у 1997 році, приніс покращені параметри форматування та таблиць.
- HTML 4.0, випущений у 1997 році, представив фрейми, таблиці стилів і сценарії.
- HTML5, випущений у 2014 році, є останньою основною версією, яка представила багато нового.

Важливість HTML у веб-розробці

- HTML є основою Інтернету та необхідним для створення веб-сторінок.
- Він забезпечує стандартну структуру, яка дозволяє веб-браузерам правильно інтерпретувати та відображати вміст.
- HTML дозволяє розробникам створювати різноманітні елементи, такі як заголовки, абзаци, зображення, посилання, форми тощо.
- Це дозволяє включати мультимедійний вміст, наприклад відео та аудіофайли.
- HTML працює в поєднанні з іншими технологіями, такими як CSS і JavaScript, щоб покращити функціональність і зовнішній вигляд веб-сторінок.
- HTML підтримується всіма основними веб-браузерами, що забезпечує сумісність між платформами.
- Це відносно проста мова для вивчення та використання, що робить її доступною як для початківців, так і для досвідчених розробників.

Важливість HTML у веб-розробці

- HTML має вирішальне значення для пошукової оптимізації (SEO), оскільки він забезпечує структуру та контекст для пошукових систем для розуміння вмісту веб-сторінки.
- Із зростанням популярності мобільних пристроїв HTML відіграє життєво важливу роль у створенні адаптивного веб-дизайну, який адаптується до різних розмірів екрана.
- HTML постійно розвивається, регулярно з'являються нові функції та вдосконалення.

Базова структура документа HTML

Документ HTML складається з комбінації тегів HTML, елементів і атрибутів.

Основна структура документа HTML включає такі компоненти:

- `<!DOCTYPE>`: визначає версію HTML і тип документа.
- `<html>`: представляє кореневий елемент сторінки HTML.
- `<head>`: містить метадані та інші невидимі елементи.
- `<title>`: визначає заголовок документа HTML, який відображається в рядку заголовка браузера.
- `<body>`: містить видимий вміст веб-сторінки

Теги, елементи та атрибути HTML

Теги: теги HTML використовуються для розмітки та визначення елементів у документі HTML. Вони взяті в кутові дужки (< і >).

Елементи: Елемент HTML складається з початкового тегу, вмісту та закриваючого тегу. Початковий тег визначає початок елемента, а закриваючий тег позначає кінець.

Атрибути: атрибути HTML надають додаткову інформацію про елемент. Вони вказуються у початковому тегу та записуються як пари ім'я-значення.

Теги, елементи та атрибути HTML

Приклад простого елемента HTML:

- Початковий тег: `<p>`
- Зміст: це елемент абзацу.
- Закриваючий тег: `</p>`
- Результат: це елемент абзацу.

DOCTYPE

- DOCTYPE означає декларацію типу документа.
- Це інструкція, яка повідомляє веб-браузеру, як інтерпретувати документ HTML.
- Оголошення DOCTYPE розміщується на початку документа HTML.
- Він визначає використовувану версію HTML і надає інформацію про тип документа.
- Оголошення DOCTYPE має вирішальне значення для забезпечення кросбраузерної сумісності та належного відтворення елементів HTML.
- Оголошення DOCTYPE допомагає веб-браузеру зрозуміти, які правила та стандарти застосовувати під час відтворення HTML-документа.
- Це гарантує, що веб-сторінка відображається правильно та узгоджено в різних браузерах і пристроях.

DOCTYPE

- Використання відповідної декларації DOCTYPE має важливе значення для сумісності з валідаторами HTML та інструментами доступності.
- Оголошення DOCTYPE також відіграє важливу роль у пошуковій оптимізації (SEO), допомагаючи пошуковим системам точно інтерпретувати вміст.

Теги заголовка (h1-h6)

- HTML надає шість рівнів тегів заголовків: h1, h2, h3, h4, h5 і h6.
- Теги заголовків використовуються для визначення заголовків і підзаголовків у документі HTML.
- Тег h1 представляє найвищий рівень заголовка, тоді як h6 представляє найнижчий рівень.
- Використання: <h1> до <h6>
- Приклад: <h1> Це заголовок</h1>

Теги абзацу, розриву рядка та горизонтальної лінії

- `<p>`: Тег абзацу використовується для визначення абзацу тексту.
 - `<br>`: Тег розриву рядка використовується для вставки розриву рядка в абзац або текст.
 - `<hr>`: Тег горизонтальної лінії використовується для створення горизонтальної лінії для візуального розділення вмісту.
-
- Використання: `<p>` для абзаців, `<br>` для розривів рядків, `<hr>` для горизонтальних ліній.
 - Приклад: `<p> Це абзац.</p><br> Це розрив рядка.<hr>`

Теги посилань і зображень

- `<a>`: Тег посилання використовується для створення гіперпосилань, що дозволяє користувачам переходити між веб-сторінками.
- Використання: `<a href="url">текст посилання</a>`
- Приклад: `<a href="https://www.example.com">Відвідайте приклад</a>`

- `<img>`: тег зображення використовується для вставки зображень у документ HTML.
- Використання: ``
- Атрибут `src` визначає URL-адресу файлу зображення.
- Атрибут `alt` надає альтернативний текст, який відображається, якщо зображення неможливо завантажити.
- Приклад: ``

- Зображення також можуть бути гіперпосиланнями, обернувши тег `<img>` у тег `<a>`.
- Приклад: `<a href="https://www.example.com"></a>`

Невпорядковані та впорядковані списки

- HTML надає два типи списків: невпорядковані списки та впорядковані списки.
- Невпорядковані списки використовуються для створення маркованих списків.
- Упорядковані списки використовуються для створення пронумерованих або буквених списків.
- Використання: `<ul>` для невпорядкованих списків, `<ol>` для впорядкованих списків.

Приклад:

Невпорядкований список	Упорядкований список
<pre> Item 1 Item 2 Item 3 </pre>	<pre> First item Second item Third item </pre>

Списки описів

- Списки описів використовуються для створення списків із термінами та їхніми відповідними описами.
- Використання: `<dl>` для списку описів, `<dt>` для терміна та `<dd>` для опису.

Приклад:

```
<dl>  
<dt>Термін 1</dt>  
<dd>Опис 1</dd>  
<dt>Сермін 2</dt>  
<dd>Опис 2</dd>  
<dt>Термін 3</dt>  
<dd>Опис3</dd>
```

Таблиці, рядки, стовпці та заголовки

- Таблиці використовуються для організації даних у рядки та стовпці.
- Структура таблиці включає рядки таблиці (`<tr>`), комірки даних таблиці (`<td>`) і заголовки таблиці (`<th>`).
- `<td>` використовується для звичайних клітинок даних, тоді як `<th>` використовується для клітинок заголовка.
- Використання:
 - a. `<table>` для контейнера таблиці.
 - b. `<tr>` для рядків таблиці.
 - в. `<td>` для звичайних клітинок даних.
 - d. `<th>` для клітинок заголовка.

Таблиці, рядки, стовпці та заголовки

Приклад (таблиці):

```
<таблиця>
```

```
<tr>
```

```
<th>Заголовок 1</th>
```

```
<th>Заголовок 2</th>
```

```
</tr>
```

```
<tr>
```

```
<td>Дані 1</td>
```

```
<td>Дані 2</td>
```

```
</tr>
```

```
<tr>
```

```
<td>Дані 3</td>
```

```
<td>Дані 4</td>
```

```
</tr>
```

Тег форми та його атрибути

- Тег `<form>` використовується для створення форми в HTML.
- Форми використовуються для збору введених даних користувачами та їх відправлення на сервер для обробки.
- Тег `<form>` має різні атрибути, які контролюють його поведінку та вигляд.
- Зазвичай використовувані атрибути форми включають:
 - а. дія: вказує URL-адресу або серверний сценарій, який оброблятиме дані форми.
 - б. метод: визначає метод HTTP, який буде використовуватися під час надсилання форми (наприклад, GET або POST).
 - в. `enctype`: визначає тип кодування для даних форми під час надсилання (наприклад, "application/x-www-form-urlencoded" або "multipart/form-data").

Типи введення

- Тег `<input>` використовується для створення поля введення у формі.
- Атрибут `type` визначає тип поля введення.
- Приклад: `<input type="text" name="username" placeholder="Введіть своє ім'я користувача">`

- Перемикачі та прапорці дозволяють користувачам вибрати один або кілька параметрів відповідно.
- **Перемикачі:**
 - a. Приклад: `<input type="radio" name="gender" value="male"> Male`
 - b. Приклад: `<input type="radio" name="gender" value="female"> Female`

Типи введення

- Прапорці:
 - a. Приклад: `<input type="checkbox" name="interest" value="music">` Музика
 - b. Приклад: `<input type="checkbox" name="interest" value="sports">` Спорт
- Тип введення надсилання використовується для створення кнопки надсилання у формі.
- Якщо натиснути, дані форми надсилаються на вказаний серверний сценарій.
- Приклад: `<input type="submit" value="Submit">`

Мітки, набір полів і легенди

- Мітки використовуються для надання текстового опису для елементів форми.
- Вони покращують доступність форми та зручність використання.
- Приклад: `<label for="username">Ім'я користувача:</label><input type="text" id="username" name="username">`
- Набір полів і легенди використовуються для групування пов'язаних елементів форми та надання заголовка для групи.
- Приклад:

```
<fieldset>
```

```
<legend>Personal Information</legend>
```

```
<!-- Form elements go here -->
```

```
</fieldset>
```



Веб-сервер Apache

Вступ до веб-сервера Apache

HTTP-сервер Apache, широко відомий як Apache, є популярним програмним забезпеченням веб-сервера з відкритим кодом. Він розроблений і підтримується Apache Software Foundation.

- Apache широко використовується для обслуговування статичного та динамічного веб-вмісту в Інтернеті.
- Apache став найпоширенішим програмним забезпеченням для веб-серверів у світі.
- Він відомий своєю стабільністю, безпекою та можливістю розширення.
- Apache підтримує кілька платформ, включаючи Linux, Unix, Windows і macOS.
- Його можна налаштувати та інтегрувати з різними мовами програмування, модулями та фреймворками.
- Модульна архітектура Apache дозволяє адміністраторам увімкнути або вимкнути певні функції залежно від їхніх потреб.

Історія веб-сервера Apache

- Apache виник у Національному центрі суперкомп'ютерних застосувань (NCSA) у середині 1990-х років.
- Початкова версія під назвою "NCSA HTTPd" була розроблена Робом МакКулом.
- У 1995 році група розробників заснувала Apache Group і випустила HTTP-сервер Apache на основі коду NCSA.
- Apache швидко набув популярності завдяки своїй стабільності, продуктивності та відкритому коду.

Основні функції веб-сервера Apache

Деякі ключові функції Apache включають:

- a. Сумісність з HTTP/1.1: підтримує останні стандарти протоколу HTTP.
- b. Віртуальний хостинг: дозволяє розміщувати кілька веб-сайтів на одному сервері.
- в. Безпека: надає різні модулі безпеки та функції, у тому числі Шифрування SSL/TLS.
- d. Продуктивність: оптимізовано для високої продуктивності та масштабованості.
- е. Розширюваність: пропонує широкий спектр модулів і API для налаштування.

Архітектура веб-сервера Apache

Apache має модульну архітектуру, яка дозволяє обробляти одночасні з'єднання та ефективно обслуговувати веб-вміст. Основними компонентами архітектури Apache є **багатопроцесорні модулі (MPM)**:

- MPM відповідають за створення та обробку процесів/потоків Apache для обробки запитів клієнтів.
- В Apache доступні різні MPM, що дозволяє адміністраторам вибрати найбільш підходящий на основі своїх потреб

Worker MPM & Prefork

Worker MPM:

- Worker MPM — це потоковий MPM, доступний в Apache.
- Він використовує кілька потоків у кожному процесі для одночасної обробки запитів клієнтів.
- Кожен потік може обслуговувати декілька запитів одночасно, що забезпечує ефективне використання ресурсів.

Prefork:

- Prefork MPM — це процесний MPM, доступний в Apache.
- Він створює кілька незалежних процесів, кожен з яких може обробляти один запит клієнта за раз.
- Prefork MPM забезпечує підвищену стабільність та ізоляцію, але може споживати більше ресурсів порівняно з Worker MPM.

Event MPM

- Event MPM — це гібридний MPM, представлений у Apache 2.4.
- Він поєднує в собі переваги MPM Worker і Prefork.
- Event MPM використовує пул потоків для обробки клієнтських запитів і розділяє обробку підключень і обробку запитів, що призводить до покращення продуктивності та ефективності ресурсів.

Архітектура на основі процесів і потоків

У архітектурі, заснованій на процесах, кожен процес обробляє по одному запиту клієнта.	У потоковій архітектурі декілька потоків у процесі обробляють одночасні запити клієнта.
Архітектура на основі процесів забезпечує кращу ізоляцію та стабільність, але може споживати більше ресурсів через накладні витрати на створення процесу.	Архітектури на основі потоків пропонують краще використання ресурсів і масштабованість, але вимагають обережного поводження для забезпечення безпеки потоків.

Конфігурація Apache

- Конфігурація Apache передбачає налаштування різних параметрів і параметрів для налаштування поведінки веб-сервера.
- Основним конфігураційним файлом для Apache є `httpd.conf`.

Файл `httpd.conf`

- Файл `httpd.conf` є основним файлом конфігурації для Apache.
- Він містить глобальні налаштування, які застосовуються до всього веб-сервера.
- Розташування файлу `httpd.conf` може відрізнятися залежно від дистрибутива Linux, але зазвичай його можна знайти в каталозі конфігурації Apache.

Конфігурація Apache

Директиви

- Директиви є основними будівельними блоками конфігурації Apache.
- Вони визначають різні налаштування та поведінку веб-сервера.
- Директиви визначено у файлі `httpd.conf` за допомогою спеціального синтаксису.

Секції та контейнери

- Розділи використовуються для групування споріднених директив і визначення їх сфери дії.
- Вони забезпечують спосіб організації та керування різними аспектами поведінки сервера.
- Загальні розділи у файлі `httpd.conf` включають розділ головного сервера, розділи віртуального хосту та розділи каталогу.

Конфігурація Apache

Файли .htaccess

- Файли htaccess є файлами конфігурації кожного каталогу.
- Вони дозволяють локальні конфігурації в межах конкретного каталогу або його підкаталогів.
- Файли .htaccess забезпечують спосіб заміни певних налаштувань Apache на основі кожного каталогу.

Приклади .htaccess

- Деякі типові випадки використання файлів .htaccess включають:
 - Налаштування контролю доступу до каталогу.
 - Увімкнення або вимкнення певних функцій.
 - URL-адреси перенаправлення.
 - Налаштування сторінок помилок.

Конфігурація Apache

Рекомендації щодо .htaccess

- Важливо використовувати файли .htaccess розумно, оскільки вони можуть вплинути на продуктивність сервера.
- Часте використання файлів .htaccess може призвести до додаткових операцій файлової системи та сповільнення.
- Рекомендується використовувати файли .htaccess лише за необхідності та розміщувати типові директиви в основному файлі httpd.conf для кращої продуктивності.

Модулі Apache

- Модулі Apache — це програмні компоненти, які розширюють функціональні можливості веб-сервера Apache.
- Модулі можуть додавати функції, покращувати продуктивність, посилювати безпеку та уможлиблювати інтеграцію з іншими технологіями.

Основні модулі

- Apache поставляється з набором основних модулів, необхідних для базового функціонування веб-сервера.
- Основні модулі забезпечують такі функції, як обробка запитів HTTP, журналювання, автентифікація та контроль доступу.
- Деякі приклади основних модулів включають `mod_rewrite`, `mod_ssl` і `mod_auth_basic`.

Модулі Apache

Додаткові та сторонні модулі

- Додаткові модулі — це додаткові модулі, які поширюються разом із Apache, але не вважаються основними.
- Вони надають додаткові функції, які можуть знадобитися не всім інсталяціям.
- Сторонні модулі розробляються спільнотою Apache або зовнішніми розробниками і не поширюються разом з Apache за умовчанням.

Деякі популярні додаткові модулі включають:

- a. `mod_proxy`: вмикає зворотне проксі та балансування навантаження.
- b. `mod_php`: інтегрує мову сценаріїв PHP із Apache.
- c. `mod_deflate`: забезпечує стиснення gzip для зменшення розміру переданих даних.
- d. `modsecurity`: підвищує безпеку веб-сервера шляхом застосування різних правил безпеки.

Активация та налаштування модулів Apache

Активация

- Щоб активувати модуль, його потрібно завантажити в конфігурацію Apache.
- Apache надає директиву LoadModule для динамічного завантаження модулів.
- Директива вказує назву модуля та шлях до спільного об'єктного файлу модуля (.so).

Конфігурація модуля

- Після активації модуля його поведінку можна налаштувати за допомогою директив конфігурації для конкретного модуля.
- Директиви конфігурації модуля вказані у файлах конфігурації Apache.
- Різні модулі мають різні параметри конфігурації, і їх можна використовувати для точного налаштування поведінки модуля.

Конфігурація модулів Apache

Деякі приклади директив конфігурації для окремих модулів включають:

- `mod_rewrite`: `RewriteRule`, `RewriteCond` для перезапису URL-адреси.
- `mod_ssl`: `SSLCertificateFile`, `SSLCipherSuite` для налаштування SSL/TLS.
- `mod_proxy`: `ProxyPass`, `ProxyPassReverse` для зворотного проксі.

Інструменти керування Apache

- Управління Apache включає різні завдання для контролю та моніторингу веб-сервера Apache.
- Ці завдання включають запуск, зупинку та перезапуск сервера, керування журналами та отримання статусу та інформації про сервер.

Інструменти керування Apache

Запуск, зупинка, перезапуск Apache

- Apache можна запускати, зупиняти та перезапускати за допомогою інструментів командного рядка або утиліт керування службами.
- У більшості дистрибутивів Linux використовується команда «`sudo systemctl start|stop|restart apache2`».
- Інші команди, як-от «`apachectl start|stop|restart`» або «`service apache2 start|stop|restart`», можуть використовуватися залежно від дистрибутива.

Інструменти керування Apache

Журнали помилок і журнали доступу

- Apache створює журнали помилок і журнали доступу для запису пов'язаних із сервером помилок і запитів клієнтів.
- Журнали помилок надають інформацію про проблеми, що виникли під час роботи сервера.
- Журнали доступу фіксують деталі запитів клієнтів, включаючи IP-адреси, запитувані файли та коди стану HTTP.

Інструменти керування Apache

Розташування журналу помилок

- Розташування файлу журналу помилок може відрізнятися залежно від дистрибутива Linux і конфігурації Apache.
- Загальні розташування включають `'/var/log/apache2/error.log'` або `'/var/log/httpd/error_log'`.
- Файли журналу помилок можна переглядати за допомогою текстових редакторів або інструментів командного рядка, таких як «tail» або «less».

Інструменти керування Apache

Розташування журналу доступу

- Розташування файлу журналу доступу також залежить від дистрибутива Linux і конфігурації Apache.
- Загальні розташування включають `'/var/log/apache2/access.log'` або `'/var/log/httpd/access_log'`.
- Файли журналів Access можна аналізувати за допомогою різних інструментів аналізу журналів або переглядати за допомогою текстових редакторів.

Інструменти керування Apache

Статус сервера та інформація

- Apache надає вбудовані функції для отримання інформації про стан сервера.
- Модуль `mod_status` можна ввімкнути для доступу до сторінки стану, де відображаються відомості про поточні з'єднання, запити та продуктивність сервера.
- Сторінка з інформацією про сервер містить вичерпну інформацію про конфігурацію сервера Apache

Інструменти керування Apache

Сторінка стану сервера

- Сторінка стану сервера надає інформацію про сервер у реальному часі
- продуктивність і зв'язки.
- До нього можна отримати доступ, відвідавши URL-адресу сервера, після якої вказано '/server status', наприклад, 'http://localhost/server-status'.
- На сторінці стану відображаються такі деталі, як поточні запити, використання ЦП, використання пам'яті та інформація про робочого/потік.

Інструменти керування Apache

Сторінка інформації про сервер

- Інформаційна сторінка сервера містить детальну інформацію про конфігурацію сервера Apache.
- До нього можна отримати доступ, відвідавши URL-адресу сервера, після якої вказано '/server info', наприклад, 'http://localhost/server-info'.
- Інформаційна сторінка показує версію сервера, завантажені модулі, змінні сервера тощо.

Віртуальні хости Apache

- Віртуальні хости Apache дозволяють розміщувати декілька веб-сайтів на одному сервері.
- Кожен віртуальний хост має власну конфігурацію та може мати унікальне доменне ім'я або IP-адресу.
- Віртуальні хости дозволяють серверу обслуговувати різний вміст на основі запитаного імені домену або IP-адреси.
- Це дозволяє розміщувати кілька веб-сайтів на одному фізичному сервері.

Типи віртуальних хостів Apache

Віртуальні хости на основі IP

- Віртуальні хости на основі IP використовують різні IP-адреси, призначені серверу, щоб розрізнити веб-сайти.
- Кожна IP-адреса пов'язана з певним веб-сайтом, що дозволяє Apache обслуговувати відповідний вміст на основі IP-адреси, використаної в запиті.

Віртуальні хости на основі імен

- Віртуальні хости на основі імен використовують запитане ім'я домену, щоб розрізнити веб-сайти.
- Apache зіставляє ім'я домену, вказане в заголовку запиту, із налаштованими віртуальними хостами та відповідно обслуговує вміст.
- Віртуальні хости на основі імен є найпоширенішим типом віртуальних хостів.

Конфігурація віртуального хосту Apache

- Віртуальні хости налаштовуються у файлах конфігурації Apache.
- Кожен віртуальний хост має власний блок конфігурації, який містить директиви, специфічні для цього хоста.
- Основний файл конфігурації (httpd.conf) або окремі файли, включені в нього, можна використовувати для визначення віртуальних хостів.

Приклади:

Директива `<VirtualHost>` використовується для визначення віртуального хосту.

Директива містить IP-адресу або «*» для всіх адрес, номер порту та ім'я домену або ім'я сервера.

У блоці `<VirtualHost>` можна визначити спеціальні директиви конфігурації для цього хоста.

Конфігурація віртуального хосту Apache

<VirtualHost *:80>

Ім'я сервера www.example.com

DocumentRoot /var/www/example

<Каталог /var/www/example>

Параметри FollowSymLinks

AllowOverride All

Доступ дозволено для всіх

</Directory>

</VirtualHost>

Найкращі методи конфігурації віртуального хосту Apache

- Використовуйте описові імена для віртуальних хостів, щоб легко визначити їх призначення.
- Переконайтеся, що записи DNS для доменних імен, які використовуються у віртуальних хостах на основі імен, налаштовано належним чином.
- Регулярно тестуйте та перевіряйте конфігурацію віртуальних хостів, щоб забезпечити належну функціональність.

Знайомство з Apache Tomcat

- Apache Tomcat — це легкий автономний веб-сервер, який забезпечує середовище на основі Java для виконання веб-програм Java.
- Він діє як контейнер для запуску Java-сервлетів, JavaServer Pages (JSP) і JavaServer Faces (JSF) програм.

Зв'язок між HTTP-сервером Apache і Tomcat

- Apache HTTP Server і Apache Tomcat — це два окремі проекти в рамках Apache Software Foundation.
- HTTP-сервер Apache — це веб-сервер загального призначення, який в основному використовується для обслуговування статичного вмісту.
- Tomcat, з іншого боку, є контейнером сервлетів і розроблений спеціально для виконання веб-додатків на основі Java.

Знайомство з Apache Tomcat

Як HTTP-сервер Apache і Tomcat працюють разом

- У багатьох розгортаннях HTTP-сервер Apache використовується як зовнішній сервер для обробки запитів клієнтів і надання статичного вмісту.
- Коли надходить запит на динамічну веб-програму (наприклад, сервлети або JSP), HTTP-сервер Apache може передати запит Tomcat за допомогою конектора Apache Tomcat (`mod_jk` або `mod_proxy`).

Знайомство з Apache Tomcat

Переваги використання HTTP-сервера Apache з Tomcat

- Використовуючи HTTP-сервер Apache перед Tomcat, ви можете скористатися його функціями, такими як балансування навантаження, кешування та завершення SSL/TLS.
- HTTP-сервер Apache також може ефективніше обробляти статичний вміст, знімаючи навантаження з Tomcat.
- Це налаштування забезпечує масштабовану та надійну архітектуру для обслуговування динамічних веб-додатків на основі Java.



Встановлення веб-сервера Apache

Передумови встановлення Apache

Перш ніж інсталювати Apache, переконайтеся, що виконано наступні передумови:

- Сумісна операційна система (Linux, Windows, MacOS)
- Достатні системні ресурси (ЦП, пам'ять, дисковий простір)
- Права адміністратора для встановлення програмного забезпечення

Встановлення Apache. Linux

- Apache можна встановити в Linux за допомогою таких менеджерів пакетів, як apt, yum або dnf.
- Ім'я пакета зазвичай "httpd" або "apache2".

Приклад команди: «`sudo apt install apache2`» (для систем на основі Debian)

Встановлення Apache. Windows

- Apache можна встановити в Windows за допомогою офіційного дистрибутива Apache Lounge або сторонніх інсталяторів, таких як XAMPP або WampServer.
- Завантажте інсталяційний пакет з офіційного сайту або сторонніх джерел.
- Дотримуйтеся вказівок майстра встановлення та виберіть потрібні параметри.

Встановлення Apache. MacOS

- Apache попередньо встановлено на MacOS. Однак для цього може знадобитися додаткове налаштування.
- Щоб увімкнути Apache на MacOS, відкрийте термінал і виконайте команду «`sudo apachectl start`».
- Перевірте встановлення, відкривши веб-браузер і перейшовши до
- 'http://localhost'.

Перевірка встановлення Apache

- Після інсталяції перевірте встановлення Apache, перейшовши на стандартну веб-сторінку Apache.
- Відкрийте веб-браузер і в адресному рядку введіть «http://localhost» або «http://<ip-адреса вашого сервера>».
- Якщо Apache працює правильно, ви повинні побачити веб-сторінку Apache за умовчанням.

Налаштування Apache

- Apache можна налаштувати відповідно до конкретних вимог.
- Зміни конфігурації вносяться у файл `httpd.conf`, який є основним файлом конфігурації для Apache.
- Файл '`httpd.conf`' знаходиться в каталозі '`conf`' інсталяції Apache.
- Скористайтесь текстовим редактором, щоб відкрити файл (наприклад, «`sudo nano /etc/httpd/conf/httpd.conf`» у Linux).
- Внесіть необхідні зміни в конфігурацію відповідно до вимог.

Налаштування Apache

Загальні зміни конфігурації

- `ServerRoot`: визначає кореневий каталог для встановлення Apache.
- `DocumentRoot`: вказує каталог, де зберігаються файли веб-сайту.
- `Port`: налаштовує номер порту, на якому Apache прослуховує вхідні з'єднання.
- `DirectoryIndex`: вказує файл за замовчуванням для обслуговування, коли запитується каталог.

Конфігурація віртуального хоста

- Віртуальні хости можна налаштувати у файлі `httpd.conf` для розміщення кількох веб-сайтів на одному сервері.
- Використовуйте директиву `<VirtualHost>`, щоб визначити кожен віртуальний хост і вказати необхідну конфігурацію всередині директиви

Налаштування Apache

Тестування змін конфігурації

- Після внесення змін у конфігурацію важливо протестувати зміни, щоб переконатися, що вони працюють належним чином.
- Використовуйте команду "`sudo apachectl configtest`" у Linux, щоб перевірити синтаксис конфігурації.
- Якщо перевірка пройшла успішно, перезапустіть Apache, щоб застосувати зміни.

Модулі Apache

Як увімкнути/вимкнути модулі

- Модулі Apache можна увімкнути або вимкнути за допомогою команд «a2enmod» і «a2dismod» у Linux.
- Приклад: «sudo a2enmod ssl», щоб увімкнути модуль SSL.
- Після ввімкнення або вимкнення модуля Apache потрібно перезапустити, щоб зміни набули чинності.

Віртуальні хости Apache

Створення та налаштування віртуальних хостів

Крок 1. Створіть новий файл конфігурації для віртуального хосту в каталозі «sites-available» (наприклад, «`sudo nano /etc/apache2/sites available/example.com.conf`»).

Крок 2. Налаштуйте віртуальний хост, вказавши домен, корінь документа, файли журналів та інші необхідні параметри.

Крок 3. Увімкніть віртуальний хост, створивши символічне посилання на каталог «sites enabled» (наприклад, «`sudo ln -s /etc/apache2/sites available/example.com.conf /etc/apache2/sitesenabled/`»).

Крок 4. Перезапустіть Apache, щоб застосувати зміни.

Віртуальні хости Apache

Усунення поширених проблем

Проблема: віртуальний хост не працює

- Перевірте, чи файл конфігурації знаходиться в каталозі «sites-available» і чи вірне його символічне посилання на «sites-enabled».
- Переконайтеся, що доменне ім'я розв'язується з правильною IP-адресою.

Проблема: помилки дозволу заборонено

- Переконайтеся, що кореневий каталог документа та інші каталоги мають необхідні дозволи для доступу Apache до файлів і їх обслуговування.
- Перевірте право власності на файл і каталог, а також дозволи за допомогою 'ls -l'

Віртуальні хости Apache

Усунення поширених проблем

Проблема: неправильна конфігурація DNS

- Переконайтеся, що доменне ім'я правильно налаштовано в налаштуваннях DNS і має правильну IP-адресу.

Проблема: конфлікт віртуальних хостів

- Перевірте, чи немає конфігурацій віртуальних хостів, які збігаються або конфліктують.
- Переконайтеся, що директиви «ServerName» і «ServerAlias» встановлено правильно для кожного віртуального хосту

Проблема: синтаксичні помилки в конфігурації віртуального хосту

- Використовуйте команду "sudo apache2ctl configtest", щоб перевірити наявність синтаксичних помилок у файлах конфігурації віртуального хосту.
- Перегляньте повідомлення про помилку та виправте всі синтаксичні помилки.

Питання та відповіді